LAPORAN PRAKTIKUM
Modul 8: HMVC, Otorisasi, dan AJAX



Disusun Oleh:

Nama : Davin Aileenskie Sofiandy

NIM : 707012500135

Kelas : D4SIKC-49-04

Mata Kuliah : Pengembangan Aplikasi Berbasis Web

TELKOM UNIVERSITY

FAKULTAS ILMU TERAPAN

JURUSAN S1 TERAPAN SISTEM INFORMASI KOTA CERDAS

TAHUN AJARAN 2026

1. Tujuan Praktikum

Praktikum Modul 8 ini saya kerjakan untuk memecah aplikasi Laravel menjadi beberapa modul dan menambahkan pembatasan akses berdasarkan peran pengguna. Materi modul memakai aplikasi perpustakaan dengan database db_sipustaka, sedangkan project yang saya pakai adalah PRK_8 dengan database hmvc_blade. Fiturnya berbeda, tetapi bagian yang dilatih sama, yaitu autentikasi manual, paket nwidart/laravel-modules, middleware role, CRUD biasa, dan CRUD berbasis AJAX. Ada beberapa tujuan yang ingin saya capai.

1. Memahami konsep HMVC di Laravel dan memasang paket nwidart/laravel-modules.

2. Memahami perbedaan autentikasi dan otorisasi, lalu menerapkan role admin dan user lewat middleware.

3. Menggunakan fitur register, login, dan logout yang berada di luar modul.

4. Membuat modul UserManagement dengan CRUD akun yang hanya bisa diakses admin dan masih memakai refresh halaman.

5. Membuat modul Pendaftaran dengan CRUD berbasis AJAX dan respons JSON yang bisa dipakai admin maupun user.

6. Menambahkan unggah berkas pada form AJAX memakai FormData dan menampilkan tautan berkasnya di tabel.

7. Memakai Form Request untuk validasi serta menampilkan pesan berhasil dan pesan error per input.

2. Alat dan Bahan

Praktikum ini saya kerjakan pada laptop pribadi dengan sistem operasi Windows 11 Pro. Sebagian besar perangkat lunak sudah tersedia dari praktikum sebelumnya. Paket nwidart/laravel-modules saya tambahkan lewat Composer, sedangkan Bootstrap, Bootstrap Icons, dan jQuery dimuat dari CDN sehingga project ini tidak membutuhkan Node.js. Aplikasi dijalankan dengan php artisan serve pada alamat http://127.0.0.1:8000.

2.1 Perangkat Lunak

Perangkat Lunak	Versi / Keterangan
XAMPP (Apache & MySQL)	Terpasang pada D:\xampp
MySQL / MariaDB	Server database melalui XAMPP, database hmvc_blade
phpMyAdmin	Untuk memeriksa tabel users dan daftars
PHP	8.2.12 (ZTS Visual C++ 2019 x64)
Composer	2.10.3
Laravel Framework	12.70.2
nwidart/laravel-modules	12.0

Bootstrap 5.3, Bootstrap Icons 1.11, jQuery 3.6	Dimuat dari CDN oleh layout view
Visual Studio Code	Editor kode dan terminal project
Git	2.53.0.windows.1
Google Chrome	Untuk menjalankan aplikasi dan memeriksa request AJAX

Bahan yang dipakai adalah project PRK_8 yang sudah berisi autentikasi manual berupa LoginController, RegisterController, dan DashboardController di folder app. Struktur folder app saya biarkan untuk enum, middleware, dan model User. Fitur baru seperti kelola user dan pendaftaran saya taruh di dalam folder Modules supaya pemisahan tiap modul terlihat jelas.

3. Langkah-Langkah Praktikum

Autentikasi dan layout dasar sudah ada dari modul sebelumnya, jadi langkah di bawah saya mulai dari pemasangan paket modul. Setelah dua modul berdiri, saya menyambungkan kolom role dan middleware ke route, lalu membangun tampilan CRUD user dan halaman pendaftaran berbasis AJAX.

3.1 Menyiapkan Database dan Memasang Paket Modul

Database hmvc_blade saya buat lewat phpMyAdmin, lalu isi .env saya sesuaikan. Nilai DB_CONNECTION diisi mysql, DB_HOST 127.0.0.1, DB_PORT 3306, DB_DATABASE hmvc_blade, dan DB_USERNAME root. DB_PASSWORD dibiarkan kosong mengikuti konfigurasi XAMPP bawaan. Paket modular ditambahkan dengan perintah composer require nwidart/laravel-modules, lalu php artisan migrate dijalankan untuk membuat tabel bawaan Laravel termasuk users.

Dua modul dibuat dengan perintah php artisan module:make UserManagement dan php artisan module:make Pendaftaran. Perintah itu menghasilkan folder Modules/UserManagement dan Modules/Pendaftaran beserta service provider, route web, route api, config, dan folder view. Status modul disimpan di modules_statuses.json. Pada project ini keduanya bernilai true, dan hasil php artisan module:list menunjukkan status Enabled untuk Pendaftaran dan UserManagement.

3.2 Menambahkan Kolom Role dan Enum UserRole

Kolom role ditambahkan ke tabel users lewat migration baru, bukan dengan mengubah migration users yang lama. Kolom itu bertipe string dengan nilai bawaan user dan diletakkan setelah kolom email. Nilai bawaan itu penting karena akun yang didaftarkan lewat halaman register otomatis menjadi user biasa tanpa perlu memilih role.

2026_11_02_add_role_to_users_table.php

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::table('users', function (Blueprint $table) {
            $table->string('role')->default('user')->after('email');
        });
    }
    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::table('users', function (Blueprint $table) {
            $table->dropColumn('role');
        });
    }
};
```

Supaya role tidak ditulis sebagai string bebas di banyak tempat, saya membuat enum UserRole di app/Enums/UserRole.php. Enum ini hanya punya dua nilai, admin dan user, serta method label() untuk menampilkan nama role di tampilan. Dengan enum, kesalahan ketik seperti Admin atau administrator bisa ditolak oleh validasi Rule::enum(UserRole::class).

app/Enums/UserRole.php

```php
<?php

namespace App\Enums;

enum UserRole: string
{
    case Admin = 'admin';
    case User = 'user';
    /**
     * Get the human readable label of the role.
     */
    public function label(): string
    {
        return match ($this) {
            self::Admin => 'Admin',
            self::User => 'User',
        };
    }
}
```

3.3 Model User

Model User saya ubah sedikit. Kolom role dimasukkan ke $fillable, diberi nilai bawaan user pada $attributes, dan di-cast ke UserRole. Cast itu membuat kode bisa membandingkan $this->role dengan UserRole::Admin, bukan dengan teks admin. Saya juga menambahkan dua helper. isAdmin() mengembalikan true hanya untuk role admin, sedangkan hasRole() menerima beberapa role sekaligus dan dipakai oleh middleware.

app/Models/User.php

```php
<?php
namespace App\Models;
use App\Enums\UserRole;
use Database\Factories\UserFactory;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Foundation\Auth\User as Authenticatable;
use Illuminate\Notifications\Notifiable;
class User

    protected $attributes = [
        'role' => 'user',
    ];

    /**
     * Get the attributes that should be cast.
     *
     * @return array<string, string>
     */
    protected function casts(): array
    {
        return [
            'email_verified_at' => 'datetime',
            'password' => 'hashed',
            'role' => UserRole::class,
        ];
    }

    public function isAdmin(): bool
    {
        return $this->role === UserRole::Admin;
    }

    public function hasRole(UserRole|string ...$roles): bool
    {
        foreach ($roles as $role) {
            if ($this->role === ($role instanceof UserRole ? $role : UserRole::tryFrom($role))) {
                return true;
            }
        }

        return false;
    }
}
```

Data awal untuk pengujian saya isi lewat DatabaseSeeder. Ada dua akun, yaitu Administrator dengan email admin@example.com berrole admin dan User Biasa dengan email user@example.com berrole user. Keduanya memakai password password. Akun admin dipakai untuk mengelola user, sedangkan akun user biasa dipakai untuk menguji penolakan akses.

3.4 Middleware EnsureUserHasRole

Pembatasan akses saya taruh di satu middleware bernama EnsureUserHasRole. Middleware ini membaca pengguna yang sedang login, lalu memanggil hasRole() dengan daftar role yang dikirim dari route. Kalau pengguna belum login atau role-nya tidak cocok, request dihentikan dengan abort(403). Pesan yang tampil di halaman galat dibuat berbahasa Indonesia supaya lebih mudah dibaca.

app/Http/Middleware/EnsureUserHasRole.php

```php
<?php

namespace App\Http\Middleware;

use Closure;
use Illuminate\Http\Request;
use Symfony\Component\HttpFoundation\Response;

class EnsureUserHasRole
{
    /**
     * Ensure the authenticated user has one of the given roles.
     *
     * Usage: ->middleware('role:admin') or ->middleware('role:admin,user')
     *
     * @param Closure(Request): (Response) $next
     */
    public function handle(Request $request, Closure $next, string ...$roles): Response
    {
        $user = $request->user();

        if ($user === null || ! $user->hasRole(...$roles)) {
            abort(403, 'Anda tidak memiliki hak akses ke halaman ini.');
        }

        return $next($request);
    }
}
```

Middleware didaftarkan pada bootstrap/app.php dengan alias role. Di sinilah parameter role:admin atau role:admin,user diteruskan ke middleware. File yang sama juga mengatur redirectGuestsTo dan redirectUsersTo supaya pengguna yang belum login diarahkan ke halaman login dan pengguna yang sudah login diarahkan ke dashboard.

bootstrap/app.php

```php
<?php

use App\Http\Middleware\EnsureUserHasRole;
use Illuminate\Foundation\Application;
use Illuminate\Foundation\Configuration\Exceptions;
use Illuminate\Foundation\Configuration\Middleware;

return Application::configure(basePath: dirname(__DIR__))
    ->withRouting(
        web: __DIR__ . '/../routes/web.php',
        commands: __DIR__ . '/../routes/console.php',
        health: '/up',
    )
    ->withMiddleware(function (Middleware $middleware): void {
        $middleware->alias([
            'role' => EnsureUserHasRole::class,
        ]);
        $middleware->redirectGuestsTo(fn () => route('login'));
        $middleware->redirectUsersTo(fn () => route('dashboard'));
    })
    ->withExceptions(function (Exceptions $exceptions): void {
        //
    })->create();
```

3.5 Modul UserManagement: Controller

Modul UserManagement dipakai admin untuk mengelola akun pengguna. Controller-nya berisi method index, create, store, edit, update, dan destroy. Method index membaca kata kunci search dan filter role dari query string, lalu membangun query dengan when(). Kata kunci dicocokkan ke nama atau email. Hasilnya diurutkan berdasarkan nama dan dibatasi sepuluh data per halaman dengan paginate(10). withQueryString() menjaga parameter pencarian tetap ada saat admin berpindah halaman.

UserManagementController.php

```php
<?php
namespace Modules\UserManagement\Http\Controllers;
use App\Enums\UserRole;
use App\Http\Controllers\Controller;
use App\Models\User;
use Illuminate\Database\Eloquent\Builder;
use Illuminate\Http\RedirectResponse;
use Illuminate\Http\Request;
use Illuminate\View\View;
use Modules\UserManagement\Http\Requests\StoreUserRequest;
use Modules\UserManagement\Http\Requests\UpdateUserRequest;
class UserManagementController

    /**
     * Display a listing of users, optionally filtered by keyword and role.
     */
    public function index(Request $request): View
    {
        $search = $request->string('search')->trim()->toString();
        $role = UserRole::tryFrom($request->string('role')->toString());

        $users = User::query()
            ->when($search !== '', function (Builder $query) use ($search) {
                $query->where(function (Builder $query) use ($search) {
                    $query->where('name', 'like', "%{$search}%")
                        ->orWhere('email', 'like', "%{$search}%");
                });
            })
            ->when($role, fn (Builder $query) => $query->where('role', $role->value))
            ->orderBy('name')
            ->paginate(10)
            ->withQueryString();

        return view('usermanagement::index', [
            'users' => $users,
            'search' => $search,
            'selectedRole' => $role,
            'roles' => UserRole::cases(),
        ]);
    }
}
```

Method update hanya mengubah password kalau kolomnya diisi. Nilai password yang kosong dibuang dari data tervalidasi sebelum $user->update(). Untuk destroy, admin tidak diizinkan menghapus akunnya sendiri. Kalau permintaan menghapus akun yang sedang aktif masuk, controller mengembalikan pesan error dan tidak menjalankan delete().

3.6 Validasi Form User

Validasi form tambah user saya pindahkan ke StoreUserRequest. Name dan email wajib diisi, email harus unik di tabel users, role wajib salah satu nilai enum, dan password wajib dikonfirmasi serta mengikuti aturan password bawaan Laravel. Method authorize() hanya mengembalikan true kalau pengguna yang mengirim form adalah admin, jadi pengguna biasa tidak bisa memakai request ini meskipun mencoba mengirim POST langsung.

StoreUserRequest.php

```php
<?php
namespace Modules\UserManagement\Http\Requests;

use App\Enums\UserRole;
use Illuminate\Foundation\Http\FormRequest;
use Illuminate\Validation\Rules\Password;

class StoreUserRequest extends FormRequest
{
    public function rules(): array
    {
        return [
            'name' => ['required', 'string', 'max:255'],
            'email' => [
                'required',
                'string',
                'email',
                'max:255',
                'unique:users,email'
            ],
            'role' => ['required', Rule::enum(UserRole::class)],
            'password' => [
                'required',
                'confirmed',
                Password::defaults()
            ],
        ];
    }
}
```

UpdateUserRequest hampir sama, tetapi email diabaikan untuk akun yang sedang diedit dan password boleh kosong. Ada satu aturan tambahan. Kalau admin mengedit akunnya sendiri, role hanya boleh admin. Aturan itu mencegah admin menurunkan role-nya sendiri sampai terkunci dari halaman kelola user. Pesan untuk aturan tersebut juga dibuat khusus berbahasa Indonesia.

UpdateUserRequest.php

```php
<?php
namespace Modules\UserManagement\Http\Requests;
use App\Enums\UserRole;
use Illuminate\Foundation\Http\FormRequest;
use Illuminate\Validation\Rule;
use Illuminate\Validation\Rules\Password;
class UpdateUserRequest

    /**
     * Get the validation rules that apply to the request.
     *
     * An admin cannot demote their own account, so they never lock themselves out.
     *
     * @return array<string, array<int, mixed>>
     */
    public function rules(): array
    {
        $user = $this->route('user');
        $isEditingOwnAccount = $user?->is($this->user()) ?? false;

        return [
            'name' => ['required', 'string', 'max:255'],
            'email' => [
                'required',
                'string',
                'email',
                'max:255',
                Rule::unique('users', 'email')->ignore($user?->id),
            ],
            'role' => [
                'required',
                Rule::enum(UserRole::class),
                Rule::when($isEditingOwnAccount, Rule::in([UserRole::Admin->value])),
            ],
            'password' => ['nullable', 'confirmed', Password::defaults()],
        ];
    }
}
```

Pada bagian yang mengatur email dan role di file yang sama, Rule::unique()->ignore() diisi id user yang sedang diedit dan Rule::when() dipakai untuk membatasi nilai role. Potongan keduanya terlihat seperti berikut.

UpdateUserRequest.php

```php
<?php

namespace Modules\UserManagement\Http\Requests;

use App\Enums\UserRole;
use Illuminate\Foundation\Http\FormRequest;
use Illuminate\Validation\Rule;

return [
            'email' => [
                'required',
                'string',
                'email',
                'max:255',
                Rule::unique('users', 'email')->ignore($user?->id),
            ],
            'role' => [
                'required',
                Rule::enum(UserRole::class),
                Rule::when($isEditingOwnAccount, Rule::in([UserRole::Admin->value])),
            ],
            'password' => ['nullable', 'confirmed', Password::defaults()],
        ];
```

3.7 Route dan Tampilan Modul UserManagement

Route pada Modules/UserManagement/routes/web.php dibungkus middleware auth dan role:admin, memakai prefix admin, dan memakai awalan nama admin. Di dalamnya hanya ada satu baris Route::resource('users'). Resource itu dibuat dengan ->except('show') karena halaman detail user tidak diperlukan. Hasilnya, route index berada di /admin/users dan nama route-nya admin.users.index.

Bagian CRUD user ini sengaja masih memakai cara biasa, yaitu form mengirim request lalu halaman dimuat ulang. View index menampilkan form pencarian di bagian atas tabel. Input search dan select role disimpan dalam satu form GET. Pilihan role langsung dikirim begitu diganti karena select memakai onchange="this.form.submit()". Tombol Reset hanya muncul kalau ada kata kunci atau filter yang sedang aktif.

index.blade.php

```blade
<form method="GET" action="{{ route('admin.users.index') }}" class="row g-2 align-items-center">
    <div class="col-sm-6 col-lg-4">
        <div class="input-group">
            <span class="input-group-text bg-white"><i class="bi bi-search"></i></span>
            <input type="search" name="search" value="{{ $search }}" class="form-control border-start-0" placeholder="Cari nama atau email...">
        </div>
    </div>
    <div class="col-sm-3 col-lg-2">
        <select name="role" class="form-select" onchange="this.form.submit()" aria-label="Filter role">
            <option value="">Semua role</option>
            @foreach ($roles as $role)
                <option value="{{ $role->value }}" @selected($selectedRole === $role)>{{ $role->label() }}</option>
            @endforeach
        </select>
    </div>
    <div class="col-auto">
        <button type="submit" class="btn btn-light border">Cari</button>
        @if ($search !== '' || $selectedRole)
            <a href="{{ route('admin.users.index') }}" class="btn btn-link text-decoration-none">Reset</a>
        @endif
    </div>
</form>
```

Isi tabel diambil dari variabel $users hasil pagination. Nomor urut dan tombol aksi saya atur supaya akun yang sedang login diberi badge Anda dan tombol Hapusnya disembunyikan. Untuk baris lain, tombol Hapus berada di dalam form yang memakai @method('DELETE') dan atribut data-confirm. Atribut itu dibaca oleh script global untuk menampilkan modal konfirmasi sebelum data benar-benar dihapus.

index.blade.php

```blade
            <tbody>
                @forelse ($users as $user)
                    @php($isCurrentUser = $user->is(auth()->user()))
                    <tr>
                        <td class="text-muted">{{ $users->firstItem() + $loop->index }}</td>
```

Form tambah dan edit sama-sama memakai partial usermanagement::partials.form supaya aturan tampilan tidak terduplikasi. Halaman create memanggil partial itu dengan $user null, sedangkan halaman edit mengirim data user yang sedang diedit.

create.blade.php

```blade
@extends('layouts.app')
@section('title', 'Tambah User')
@section('content')
<div class="row justify-content-center">
    <div class="col-lg-7">
        <a href="{{ route('admin.users.index') }}" class="text-decoration-none small">
            <i class="bi bi-arrow-left"></i> Kembali ke daftar user
        </a>

        <h4 class="mt-2 mb-4">Tambah User</h4>

        <div class="card">
            <div class="card-body p-4">
                <form action="{{ route('admin.users.store') }}" method="POST">
                    @csrf

                    @include('usermanagement::partials.form', ['user' => null])

                    <div class="d-flex justify-content-end gap-2">
                        <a href="{{ route('admin.users.index') }}" class="btn btn-light">Batal</a>
                        <button type="submit" class="btn btn-primary">Simpan</button>
                    </div>
                </form>
            </div>
        </div>
    </div>
</div>
@endsection
```

3.8 Modul Pendaftaran: Migration, Model, dan Route

Dua migration yang menangani role dan tabel daftars dibuat terpisah. Migration kolom role memakai string('role')->default('user')->after('email'), sehingga akun lama otomatis menjadi user. Migration create_daftars_table membuat tabel daftars dengan email unik dan no_hp sepanjang 20 karakter, sedangkan migration add_berkas_to_daftars_table menambahkan kolom berkas dan nama_berkas yang boleh kosong.

create_daftars_table.php

```php
<?php
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('daftars', function (Blueprint $table) {
            $table->id();
            $table->string('nama');
            $table->string('asal_sekolah');
            $table->string('prodi_tujuan');
            $table->string('email')->unique();
            $table->string('no_hp', 20);
            $table->timestamps();
        });
    }
    /**
     * Reverse the migrations.
     */
};
```

Model Daftar mendaftarkan semua kolom ke $fillable. Ada accessor berkasUrl yang otomatis ikut di JSON dan mengubah path file menjadi URL asset('storage/' . berkas). Dengan accessor itu, JavaScript cukup memakai item.berkas_url untuk menautkan file, tanpa perlu merangkai URL sendiri di sisi browser.

Models/Daftar.php

```php
<?php
namespace Modules\Pendaftaran\Models;
use Illuminate\Database\Eloquent\Casts\Attribute;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
use Modules\Pendaftaran\Database\Factories\DaftarFactory;
class Daftar

    /**
     * The attributes that are mass assignable.
     *
     * @var list<string>
     */
    protected $fillable = [
        'nama',
        'asal_sekolah',
        'prodi_tujuan',
        'email',
        'no_hp',
        'berkas',
        'nama_berkas',
    ];

    /**
     * The accessors to append to the model's array and JSON form.
     *
     * @var list<string>
     */
    protected $appends = [
        'berkas_url',
    ];

    /**
     * Public URL of the uploaded file, or null when there is no file.
     */
    protected function berkasUrl(): Attribute
    {
        return Attribute::get(fn (): ?string => $this->berkas ? asset('storage/' . $this->berkas) : null);
    }
}
```

Route pendaftaran berada di Modules/Pendaftaran/routes/web.php dan dibungkus middleware auth serta role:admin,user. Route-nya ditulis satu per satu, bukan resource, karena ada route tambahan pendaftaran/data yang mengembalikan JSON. Hasil php artisan route:list -v menunjukkan enam route pendaftaran dengan middleware web, auth, dan role:admin,user.

Modules/Pendaftaran/routes/web.php

```php
<?php

use Illuminate\Support\Facades\Route;
use Modules\Pendaftaran\Http\Controllers\PendaftaranController;

Route::middleware(['auth', 'role:admin,user'])->group(function () {
    Route::get('pendaftaran', [
        PendaftaranController::class,
        'index'
    ])->name('pendaftaran.index');
    Route::get('pendaftaran/data', [
        PendaftaranController::class,
        'getData'
    ])->name('pendaftaran.data');
    Route::post('pendaftaran', [
        PendaftaranController::class,
        'store'
    ])->name('pendaftaran.store');
    Route::get(
        'pendaftaran/{daftar}/edit',
        [PendaftaranController::class, 'edit']
    )->name('pendaftaran.edit');
    Route::put('pendaftaran/{daftar}', [
        PendaftaranController::class,
        'update'
    ])->name('pendaftaran.update');
    Route::delete(
        'pendaftaran/{daftar}',
        [PendaftaranController::class, 'destroy']
    )->name('pendaftaran.destroy');
});
```

3.9 Controller Pendaftaran

PendaftaranController mengembalikan view pada index, sedangkan data diambil terpisah lewat getData(). Method getData() mengembalikan Daftar::latest()->get() sebagai JSON. Method store menyimpan data tervalidasi selain berkas, lalu menyimpan file ke disk public pada folder berkas_pendaftaran dan menyimpan nama aslinya di kolom nama_berkas. Method edit mengembalikan satu data untuk mengisi form edit.

PendaftaranController.php

```php
<?php
namespace Modules\Pendaftaran\Http\Controllers;

use Illuminate\Http\JsonResponse;
use Modules\Pendaftaran\Http\Requests\DaftarRequest;
use Modules\Pendaftaran\Models\Daftar;

class PendaftaranController extends Controller
{
    public function getData(): JsonResponse
    {
        return response()->json(Daftar::latest()->get());
    }

    /**
     * Store a newly created registration together with its uploaded file.
     */
    public function store(DaftarRequest $request): JsonResponse
    {
        $data = $request->safe()->except('berkas');

        $file = $request->file('berkas');
        $data['berkas'] = $file->store('berkas_pendaftaran', 'public');
        $data['nama_berkas'] = $file->getClientOriginalName();

        Daftar::create($data);

        return response()->json(['success' => 'Data berhasil disimpan.']);
    }

    /**
     * Return a single registration for the edit form.
     */
    public function edit(Daftar $daftar): JsonResponse
    {
        return response()->json($daftar);
    }
}
```

Method update hanya mengganti file kalau ada file baru. File lama dihapus dari disk setelah file baru berhasil disimpan, sehingga data tidak kehilangan path kalau upload gagal. Method destroy menghapus file fisik lebih dulu, lalu menghapus barisnya. Semua method tulis mengembalikan JSON berisi kunci success atau validation error dengan status 422, bukan redirect halaman.

3.10 Validasi Form Pendaftaran

Validasi form pendaftaran ada di DaftarRequest. Semua isian teks wajib diisi, email harus unik di tabel daftars, dan no_hp harus cocok dengan regex ^[0-9+]{8,20}$. Aturan berkas bergantung pada method request. Saat membuat data baru berkas wajib diunggah, sedangkan saat mengubah data berkas boleh kosong supaya file lama tetap dipakai. Jenis file dibatasi ke jpg, jpeg, png, dan pdf dengan ukuran maksimal 2 MB.

DaftarRequest.php

```php
<?php
namespace Modules\Pendaftaran\Http\Requests;
use Illuminate\Foundation\Http\FormRequest;
use Illuminate\Validation\Rule;
class DaftarRequest

    /**
     * Get the validation rules that apply to the request.
     *
     * The file is required when creating, and optional when updating (keep the old file).
     *
     * @return array<string, array<int, mixed>>
     */
    public function rules(): array
    {
        $isUpdate = $this->isMethod('PUT') || $this->isMethod('PATCH');
        $daftar = $isUpdate ? $this->route('daftar') : null;

        return [
            'nama' => ['required', 'string', 'max:255'],
            'asal_sekolah' => ['required', 'string', 'max:255'],
            'prodi_tujuan' => ['required', 'string', 'max:255'],
            'email' => [
                'required',
                'email',
                'max:255',
                Rule::unique('daftars', 'email')->ignore($daftar?->id),
            ],
            'no_hp' => ['required', 'string', 'regex:/^[0-9+]{8,20}$/'],
            'berkas' => [$isUpdate ? 'nullable' : 'required', 'file', 'mimes:jpg,jpeg,png,pdf', 'max:2048'],
        ];
    }

    /**
     * Get custom messages for validator errors.
     *
     * @return array<string, string>
     */
    public function messages(): array
    {
        return [
            'no_hp.regex' => 'No HP hanya boleh berisi angka (8-20 digit).',
            'berkas.required' => 'Berkas wajib diunggah.',
            'berkas.mimes' => 'Berkas harus berformat jpg, jpeg, png, atau pdf.',
            'berkas.max' => 'Ukuran berkas maksimal 2 MB.',
        ];
    }
}
```

Pesan validasi dibuat berbahasa Indonesia, misalnya No HP hanya boleh berisi angka (8-20 digit) dan Berkas wajib diunggah. Pengabaian aturan email unik memakai Rule::unique()->ignore() dengan id data yang sedang diedit, jadi email pendaftar yang tidak diubah tetap lolos validasi.

3.11 Tampilan AJAX Modul Pendaftaran

Halaman pendaftaran memakai modal Bootstrap untuk form tambah dan edit. Di bagian paling awal view, saya mendefinisikan objek urls yang berisi route data, store, edit, update, dan destroy. Route edit dan update memakai placeholder :id yang diganti id data saat tombol Edit ditekan. Token CSRF dan header Accept diatur sekali lewat $.ajaxSetup(), sehingga setiap request AJAX otomatis membawa keduanya.

index.blade.php

```
    const urls = {
        data: "{{ route('pendaftaran.data') }}",
        store: "{{ route('pendaftaran.store') }}",
        edit: "{{ route('pendaftaran.edit', ':id') }}",
        update: "{{ route('pendaftaran.update', ':id') }}",
        destroy: "{{ route('pendaftaran.destroy', ':id') }}",
    };

    let records = [];

    // Set CSRF token untuk semua request AJAX
    $.ajaxSetup({
        headers: {
            'X-CSRF-TOKEN': $('meta[name="csrf-token"]').attr('content'),
            'Accept': 'application/json'
        }
    });
```

Data dari server dirender ke tabel lewat JavaScript. Saya menambahkan fungsi escapeHtml() untuk mengubah karakter &, <, >, " dan ' menjadi entity HTML sebelum data dimasukkan ke template. Tanpa fungsi itu, nama pendaftar yang berisi kode script bisa ikut dieksekusi di browser. Fungsi fileLink() memakai berkas_url untuk menampilkan ikon PDF atau gambar sesuai ekstensi nama file.

index.blade.php

```
    // Escape karakter HTML agar data user tidak dieksekusi sebagai script (XSS)
    function escapeHtml(value) {
        return String(value ?? '')
            .replace(/&/g, '&amp;')
            .replace(/</g, '&lt;')
            .replace(/>/g, '&gt;')
            .replace(/"/g, '&quot;')
            .replace(/'/g, '&#039;');
    }

    // Link untuk membuka berkas di tab baru
    function fileLink(item) {
        if (!item.berkas_url) {
            return '<span class="text-muted">-</span>';
        }
        const isPdf = (item.nama_berkas || item.berkas).toLowerCase().endsWith('.pdf');
        const icon = isPdf ? 'bi-file-earmark-pdf text-danger' : 'bi-file-earmark-image text-primary';

        return `<a href="${escapeHtml(item.berkas_url)}" target="_blank" rel="noopener" class="small text-decoration-none">
            <i class="bi ${icon} me-1"></i>${escapeHtml(item.nama_berkas || 'Lihat berkas')}
        </a>`;
    }
```

Bagian tabel HTML hanya berisi kepala kolom No, Nama, Asal Sekolah, Prodi Tujuan, Kontak, Berkas, dan Aksi. Badan tabel awalnya diisi satu baris spinner dengan teks Memuat data, supaya pengguna tahu data sedang diambil. Baris itu nanti diganti oleh hasil renderTable() setelah request data selesai.

index.blade.php

```
        <table class="table table-hover" id="pendaftaran-table">
            <thead>
                <tr>
                    <th width="5%">No</th>
                    <th>Nama</th>
                    <th>Asal Sekolah</th>
                    <th>Prodi Tujuan</th>
                    <th>Kontak</th>
                    <th>Berkas</th>
                    <th class="text-end">Aksi</th>
                </tr>
            </thead>
            <tbody>
                <tr>
                    <td colspan="7" class="text-center text-muted py-5">
                        <div class="spinner-border spinner-border-sm me-2" role="status"></div> Memuat data...
                    </td>
                </tr>
            </tbody>
        </table>
```

Fungsi renderTable() memfilter variabel records sesuai kata kunci, lalu menyusun HTML baris. Pencarian di halaman ini berjalan di browser, tanpa request ke server, karena seluruh data sudah ada di variabel records. Kalau hasil filter kosong, tabel menampilkan pesan Tidak ada pendaftar yang cocok dengan pencarian. Tombol Edit dan Hapus membawa data-id, dan tombol Hapus juga membawa data-nama untuk pesan konfirmasi.

index.blade.php

```
function renderTable() {
    const keyword = $('#search').val().trim().toLowerCase();
    const rows = records.filter((item) =>
        [item.nama, item.asal_sekolah, item.prodi_tujuan, item.email, item.no_hp]
            .some((value) => (value || '').toLowerCase().includes(keyword))
    );

    $('#total-count').text(records.length);

    if (records.length === 0) {
        $('#pendaftaran-table tbody').html(emptyRow('bi-inbox', 'Belum ada pendaftar. Klik <strong>Tambah Pendaftar</strong> untuk memulai.'));
        return;
    }

    if (rows.length === 0) {
        $('#pendaftaran-table tbody').html(emptyRow('bi-search', 'Tidak ada pendaftar yang cocok dengan pencarian.'));
        return;
    }

    const html = rows.map((item, index) => `
        <tr>
            <td class="text-muted">${index + 1}</td>
            <td class="fw-medium">${escapeHtml(item.nama)}</td>
            <td>${escapeHtml(item.asal_sekolah)}</td>
            <td><span class="badge text-bg-light border">${escapeHtml(item.prodi_tujuan)}</span></td>
            <td class="small">
                <div>${escapeHtml(item.email)}</div>
                <div class="text-muted">${escapeHtml(item.no_hp)}</div>
            </td>
            <td>${fileLink(item)}</td>
            <td class="text-end text-nowrap">
                <button data-id="${item.id}" class="edit btn btn-sm btn-outline-primary">
                    <i class="bi bi-pencil"></i> Edit
                </button>
                <button data-id="${item.id}" data-nama="${escapeHtml(item.nama)}" class="delete btn btn-sm btn-outline-danger">
                    <i class="bi bi-trash"></i> Hapus
                </button>
            </td>
        </tr>
    `).join('');

    $('#pendaftaran-table tbody').html(html);
}
```

Fungsi loadData() mengirim GET ke urls.data dan menyimpan hasilnya ke variabel records sebelum memanggil renderTable(). Kalau request gagal, badan tabel diisi pesan Gagal memuat data dan penyebabnya dicetak di console. Event input pada input pencarian memanggil renderTable() langsung tanpa request baru.

index.blade.php

```
function loadData() {
    $.ajax({
        url: urls.data,
        type: 'GET',
        dataType: 'json',
        success: function (data) {
            records = data;
            renderTable();
        },
        error: function (xhr) {
            console.error('Error:', xhr.responseText);
            $('#pendaftaran-table tbody').html(emptyRow('bi-wifi-off', 'Gagal memuat data. Silakan refresh halaman.'));
        }
    });
}
```

Form modal berisi input nama, asal sekolah, prodi tujuan, email, no hp, dan berkas. Setiap input punya elemen invalid-feedback kosong yang akan diisi pesan dari server. Input berkas diberi label bintang wajib yang disembunyikan saat mode edit, tepat seperti aturan validasi DaftarRequest.

index.blade.php

```blade
<!-- Modal form -->
<div class="modal fade" id="pendaftaranModal" tabindex="-1" aria-labelledby="pendaftaranModalLabel" aria-hidden="true">
    <div class="modal-dialog modal-dialog-centered">
        <div class="modal-content">
            <div class="modal-header">
                <h5 class="modal-title" id="pendaftaranModalLabel">Tambah Pendaftar</h5>
                <button type="button" class="btn-close" data-bs-dismiss="modal" aria-label="Tutup"></button>
            </div>
            <div class="modal-body">
                <form id="pendaftaranForm">
                    @csrf
                    <input type="hidden" name="id" id="id">

                    @foreach ([
                        'nama' => ['label' => 'Nama Lengkap', 'type' => 'text', 'placeholder' => 'Nama pendaftar'],
                        'asal_sekolah' => ['label' => 'Asal Sekolah', 'type' => 'text', 'placeholder' => 'Contoh: SMA Negeri 1 Bandung'],
                        'prodi_tujuan' => ['label' => 'Prodi Tujuan', 'type' => 'text', 'placeholder' => 'Contoh: Informatika'],
                        'email' => ['label' => 'Email', 'type' => 'email', 'placeholder' => 'nama@email.com'],
                        'no_hp' => ['label' => 'No HP', 'type' => 'tel', 'placeholder' => 'Contoh: 081234567890'],
                    ] as $name => $field)
                    <div class="mb-3">
                        <label for="{{ $name }}" class="form-label">
                            {{ $field['label'] }} <span class="text-danger">*</span>
                        </label>
                        <input type="{{ $field['type'] }}" class="form-control" id="{{ $name }}" name="{{ $name }}" placeholder="{{ $field['placeholder'] }}" required>
                        <div class="invalid-feedback" id="{{ $name }}-error"></div>
                    </div>
                    @endforeach

                    <div class="mb-3">
                        <label for="berkas" class="form-label">
                            Berkas Pendukung <span class="text-danger" id="berkas-required">*</span>
                        </label>
                        <div class="small mb-2 d-none" id="berkas-current">
                            Berkas saat ini: <a href="#" target="_blank" rel="noopener" id="berkas-current-link"></a>
                        </div>
                        <input type="file" class="form-control" id="berkas" name="berkas" accept=".jpg,.jpeg,.png,.pdf">
                        <div class="invalid-feedback" id="berkas-error"></div>
                        <div class="form-text" id="berkas-help">Format jpg, jpeg, png, atau pdf. Maksimal 2 MB.</div>
                    </div>
                </form>
            </div>
            <div class="modal-footer">
                <button type="button" class="btn btn-light" data-bs-dismiss="modal">Batal</button>
                <button type="button" class="btn btn-primary" id="saveBtn">
                    <span class="spinner-border spinner-border-sm me-1 d-none" role="status" aria-hidden="true"></span>
                    <span class="submit-text">Simpan</span>
                </button>
            </div>
        </div>
    </div>
</div>
```

Event untuk tombol Tambah Pendaftar, submit form, dan blok tombol simpan berada di bagian bawah script. Mode tambah dan mode edit dibedakan dari nilai input id yang tersembunyi. Pada mode edit, data lama diambil lewat $.get() sebelum modal ditampilkan.

index.blade.php

```blade
    // Tambah data
    $('#addBtn').click(function () {
        $('#pendaftaranModalLabel').text('Tambah Pendaftar');
        setFileMode();
        pendaftaranModal.show();
    });

    // Tekan Enter di form = simpan
    $('#pendaftaranForm').on('submit', function (e) {
        e.preventDefault();
        $('#saveBtn').trigger('click');
    });
```

Saat form dikirim, JavaScript mengirim objek FormData dengan processData dan contentType disetel false supaya file ikut terkirim. Kalau ada id, _method PUT ditambahkan ke FormData. Tombol simpan dinonaktifkan selama request berjalan dan diberi teks Menyimpan. Tanggapan 422 dipakai untuk menandai input yang salah dengan class is-invalid dan menampilkan pesan pertama untuk tiap field.

index.blade.php

```
        const formData = new FormData(document.getElementById('pendaftaranForm'));
        if (id) {
            formData.append('_method', 'PUT');
        }

        $.ajax({
            url: id ? urls.update.replace(':id', id) : urls.store,
            type: 'POST',
            data: formData,
            processData: false,
            contentType: false,
            dataType: 'json',
            success: function (response) {
                pendaftaranModal.hide();
                loadData();
                showAlert(response.success);
            },
            error: function (xhr) {
                if (xhr.status === 422) {
                    $.each(xhr.responseJSON.errors, function (key, value) {
                        $(`#${key}`).addClass('is-invalid');
                        $(`#${key}-error`).text(value[0]);
                    });
                    $('#pendaftaranForm .is-invalid:first').focus();
                } else {
                    console.error('Error:', xhr.responseText);
                    pendaftaranModal.hide();
                    showAlert('Terjadi kesalahan. Silakan coba lagi.', 'danger');
                }
            },
```

Tombol Edit memanggil route edit lewat $.get(), lalu mengisi nilai input dengan data yang diterima. Nama file lama ditampilkan sebagai tautan, jadi pengguna tahu file mana yang sedang dipakai. Tombol Hapus memakai modal konfirmasi global bernama confirmDelete() yang mengembalikan Promise, sehingga penghapusan baru dikirim setelah tombol Ya, hapus ditekan.

index.blade.php

```
    // Edit data
    $(document).on('click', '.edit', function () {
        const id = $(this).data('id');
        $.get(urls.edit.replace(':id', id), function (data) {
            $('#pendaftaranModalLabel').text('Edit Pendaftar');
            $('#id').val(data.id);
            $('#nama').val(data.nama);
            $('#asal_sekolah').val(data.asal_sekolah);
            $('#prodi_tujuan').val(data.prodi_tujuan);
            $('#email').val(data.email);
            $('#no_hp').val(data.no_hp);
            setFileMode(data);
            pendaftaranModal.show();
        }).fail(function () {
            showAlert('Gagal memuat data untuk diedit.', 'danger');
        });
    });

    // Hapus data
    $(document).on('click', '.delete', function () {
        const id = $(this).data('id');
        const nama = $(this).data('nama');

        confirmDelete(`Data pendaftar "${nama}" akan dihapus dan tidak bisa dikembalikan.`).then(function (confirmed) {
            if (!confirmed) {
                return;
            }
            $.ajax({
                type: 'DELETE',
                url: urls.destroy.replace(':id', id),
                dataType: 'json',
                success: function (response) {
                    loadData();
                    showAlert(response.success);
                },
                error: function () {
                    showAlert('Gagal menghapus data. Silakan coba lagi.', 'danger');
                }
            });
        });
    });
});
```

3.12 Layout dan Navigasi

Layout app.blade.php memakai Bootstrap dari CDN. Menu Pendaftaran selalu tampil untuk pengguna yang sudah login, sedangkan menu Kelola User hanya tampil kalau auth()->user()->isAdmin() bernilai true. Pembatasan menu di tampilan tidak menggantikan middleware. Middleware tetap menjadi penjaga sebenarnya di sisi server, sedangkan kondisi di view hanya menyembunyikan tautan yang tidak boleh dibuka.

layouts/app.blade.php

```blade
<ul class="navbar-nav me-auto ms-lg-4 gap-lg-1">
    <li class="nav-item">
        <a class="nav-link {{ request()->routeIs('dashboard') ? 'active' : '' }}" href="{{ route('dashboard') }}">
            <i class="bi bi-house me-1"></i> Dashboard
        </a>
    </li>
    <li class="nav-item">
        <a class="nav-link {{ request()->routeIs('pendaftaran.*') ? 'active' : '' }}" href="{{ route('pendaftaran.index') }}">
            <i class="bi bi-clipboard me-1"></i> Pendaftaran
        </a>
    </li>
    @if (auth()->user()->isAdmin())
    <li class="nav-item">
        <a class="nav-link {{ request()->routeIs('admin.users.*') ? 'active' : '' }}" href="{{ route('admin.users.index') }}">
            <i class="bi bi-people me-1"></i> Kelola User
        </a>
    </li>
    @endif
</ul>
```

Partial scripts.blade.php memuat jQuery dan bundle Bootstrap, lalu menyediakan tiga perilaku global. Modal konfirmasi untuk form yang punya atribut data-confirm, tombol mata untuk menampilkan atau menyembunyikan password, dan penutupan otomatis pesan sukses setelah empat detik. Blok @stack('scripts') di bagian akhir dipakai view modul untuk menyisipkan script AJAX pendaftaran.

layouts/partials/scripts.blade.php

```blade
<script>

window.confirmDelete = function (message) {
    const modalElement = document.getElementById('confirmModal');
    const modal = bootstrap.Modal.getOrCreateInstance(modalElement);

    document.getElementById('confirmModalMessage').textContent = message;

    return new Promise(function (resolve) {
        let confirmed = false;
        const yesButton = document.getElementById('confirmModalYes');

        yesButton.onclick = function () {
            confirmed = true;
            modal.hide();
        };

        modalElement.addEventListener('hidden.bs.modal', function () {
            resolve(confirmed);
        }, { once: true });

        modal.show();
    });
};
</script>
```

3.13 Menjalankan Aplikasi dan Menguji

Aplikasi dijalankan dengan php artisan serve pada alamat http://127.0.0.1:8000. Pengujian saya lakukan dalam dua tahap. Tahap pertama memakai akun admin@example.com untuk melihat menu Kelola User, menambah user, mengubah role, dan mencoba menghapus akun. Tahap kedua memakai akun user@example.com untuk memastikan menu Kelola User tidak muncul dan alamat /admin/users menghasilkan halaman 403.

Untuk bagian AJAX, saya memastikan tabel pendaftar terisi tanpa halaman berpindah, mencoba tombol Tambah Pendaftar, mengisi form dengan data valid dan tidak valid, lalu menghapus satu data. Setiap tanggapan dipantau dari pesan yang muncul di atas tabel dan dari tab Network browser.

4. Hasil dan Dokumentasi Program

Semua bagian sudah dibuat dan bisa dijalankan. Tabel users memiliki kolom role, middleware role terpasang pada route kedua modul, halaman Kelola User dijaga oleh role admin, dan halaman Pendaftaran memakai AJAX untuk seluruh operasi datanya. Bagian ini mencatat hasil yang sudah saya periksa langsung dari project.

Perlu saya sebutkan sejak awal bahwa tangkapan layar pengujian belum saya simpan. Gambar yang ada di laporan ini hanya gambar kode program pada Bab 3. Supaya Bab 4 tidak berisi klaim tanpa bukti, bagian ini hanya memuat hal yang bisa diperiksa ulang dari perintah atau dari kode.

Struktur aplikasi. modules_statuses.json berisi true untuk UserManagement dan Pendaftaran, dan hasil php artisan module:list menunjukkan keduanya Enabled. Route yang terdaftar juga sesuai rancangan. Route admin.users berjumlah enam karena resource dibuat dengan except show, sedangkan route pendaftaran berjumlah enam, yaitu pendaftaran.index, pendaftaran.data, pendaftaran.store, pendaftaran.update, pendaftaran.edit, dan pendaftaran.destroy. Hasil php artisan route:list -v memperlihatkan middleware web, auth, dan role:admin,user pada keenam route pendaftaran.

Struktur database. php artisan migrate:status menunjukkan migration 2026_11_02_012219_add_role_to_users_table, 2026_11_02_022438_create_daftars_table, dan 2026_11_02_022837_add_berkas_to_daftars_table sudah dijalankan. Seeder membuat dua akun uji, admin@example.com dengan role admin dan user@example.com dengan role user.

Alur pengujian di browser. Pertama, login sebagai admin untuk memeriksa menu Kelola User, halaman daftar akun, pencarian, dan filter role. Kedua, login sebagai user biasa lalu membuka /admin/users untuk memastikan middleware menolak dengan halaman 403. Ketiga, menambah akun, mengubah role, mencoba menghapus akun sendiri, dan menghapus akun lain lewat modal konfirmasi. Keempat, membuka /pendaftaran dan menyimpan, mengubah, serta menghapus data lewat modal AJAX, termasuk mengirim email yang sudah dipakai dan nomor HP kurang dari delapan digit. Setiap langkah itu perlu diambil tangkapan layarnya supaya Bab 4 punya bukti visual.

5. Analisis dan Pembahasan

1. Pemetaan konsep HMVC pada project ini.

HMVC di sini bukan berarti setiap modul berjalan sebagai aplikasi terpisah. Semua modul masih memakai satu database, satu layout, dan satu proses aplikasi. Yang dipisahkan adalah kode fitur. Controller, request, model, migration, dan view milik satu fitur disimpan dalam folder Modules masing-masing. Paket nwidart/laravel-modules mendaftarkan service provider tiap modul lewat modules_statuses.json, sehingga route dan view di dalamnya tetap dikenali Laravel. Keuntungannya terasa di view. Namespace usermanagement::index dan pendaftaran::index membuat nama view yang sama seperti index tidak saling menimpa.

2. Autentikasi dan otorisasi dikerjakan di dua tempat.

Autentikasi berada di luar modul. LoginController, RegisterRequest, dan route auth memakai middleware guest serta auth, dan RegisterRequest tidak memiliki field role sehingga pengguna baru selalu menjadi user. Otorisasi dikerjakan middleware role menggunakan isAdmin() dan hasRole(). Pemisahan ini penting karena modul tidak perlu tahu bagaimana pengguna masuk, cukup tahu role yang sedang aktif.

3. Pembatasan akses berlapis antara middleware, request, dan tampilan.

Middleware role:admin dan role:admin,user adalah penjaga pertama. Route yang tidak cocok langsung dihentikan dengan 403 sebelum controller dijalankan. Di sisi form request, authorize() memeriksa isAdmin() sekali lagi, sehingga aturan pada request tetap aman meskipun route suatu saat dilewati. Di view, menu Kelola User disembunyikan dengan @if (auth()->user()->isAdmin()). Kondisi itu hanya pengaturan tampilan. Kalau pengguna biasa mengetik /admin/users di address bar, yang menolak tetap middleware, bukan hilangnya menu.

4. Perbandingan CRUD sinkron dan CRUD AJAX.

CRUD user memakai cara sinkron karena halamannya sederhana dan dipakai jarang. Form mengirim POST atau PUT lewat route resource, lalu controller melakukan redirect dengan pesan session. CRUD pendaftaran memakai AJAX karena tabelnya sering berubah dan diakses admin maupun user. Setiap method tulis mengembalikan JSON, dan tabel dirender ulang dari variabel records tanpa memuat dokumen baru. Konsekuensinya, validasi pada form AJAX tidak memakai @error bawaan Blade, tetapi memakai tanggapan 422 yang dipetakan ke elemen invalid-feedback per input.

5. Alur AJAX pada modul Pendaftaran.

Pembacaan data memakai route pendaftaran.data, penyimpanan memakai pendaftaran.store, perubahan memakai pendaftaran.update dengan method PUT disisipkan lewat _method, dan penghapusan memakai pendaftaran.destroy dengan method DELETE. Karena semua request mengirim header Accept: application/json, request AJAX tidak akan menerima halaman HTML. Laravel membalas dengan JSON, baik berisi kunci success maupun daftar error validasi 422.

6. Keamanan saat data dirender dengan JavaScript.

Data dari tabel daftars berasal dari input pengguna dan dirender lewat template string JavaScript. Kalau nilai seperti nama dimasukkan langsung ke innerHTML, isi yang mengandung tag HTML atau kode script bisa berjalan di browser. Karena itu setiap nilai melewati

escapeHtml() lebih dulu, dan atribut data-nama pada tombol Hapus juga memakai fungsi yang sama. Pengaman ini tidak menggantikan validasi di server, tetapi menutup celah XSS yang muncul dari cara data ditampilkan.

7. Pengelolaan berkas pendukung pendaftaran.

Berkas disimpan di storage/app/public/berkas_pendaftaran dengan nama acak dari method store(). Database hanya menyimpan path di kolom berkas dan nama asli di kolom nama_berkas. Accessor berkasUrl mengubah path itu menjadi URL asset('storage/' . berkas), sehingga file bisa dibuka lewat symbolic link public/storage. Agar tautan itu berjalan, php artisan storage:link harus sudah dijalankan. Pada project ini folder public/storage sudah ada, jadi tautan file pendaftar bisa dibuka di tab baru dari tabel.

8. Kendala dan penyelesaian.

Kendala yang saya catat berasal dari log project. Saat mencoba mengecek isi tabel users lewat skrip sebelum MySQL dinyalakan, muncul galat SQLSTATE[HY000] [2002] No connection could be made because the target machine actively refused it pada host 127.0.0.1 port 3306 di storage/logs/laravel.log. Penyebabnya bukan kode aplikasi, melainkan server MySQL yang belum berjalan. Setelah modul MySQL dinyalakan dari XAMPP Control Panel, perintah migrate dan migrate:status berjalan normal.

Masalah kedua terjadi saat saya menyiapkan gambar kode untuk laporan. File create.blade.php pada modul UserManagement terhapus tidak sengaja oleh skrip laporan yang saya jalankan dengan folder kerja yang salah. Saya memulihkannya dengan git checkout -- pada file itu, dan isinya kembali sesuai versi terakhir di Git. Kejadian ini tidak mengubah kode aplikasi, tetapi menunjukkan pentingnya menyimpan pekerjaan di Git sebelum menjalankan skrip otomatis.

9. Perbedaan dengan modul.

Perbedaan pertama ada pada objek studinya. Modul memakai aplikasi perpustakaan dengan database db_sipustaka dan modul Pinjaman, sedangkan project ini memakai kasus pendaftaran mahasiswa baru pada database hmvc_blade. Perbedaan kedua ada pada tampilan. Project ini memakai Bootstrap dari CDN dan layout app.blade.php sendiri, bukan Breeze. Route pendaftaran ditulis satu per satu karena ada route JSON terpisah untuk data tabel, sedangkan form CRUD user memakai halaman create dan edit terpisah. Tangkapan layar pengujian pada Bab 4 juga belum tersedia, sehingga bagian itu baru berisi hasil pemeriksaan perintah. Perbedaan ini tidak mengubah bagian yang dilatih, yaitu HMVC, otorisasi role, dan AJAX.

6. Kesimpulan

Struktur modular berhasil diterapkan pada project PRK_8 dengan paket nwidart/laravel-modules. Modul UserManagement dan Pendaftaran aktif, masing-masing punya route, controller, request, model, dan view sendiri, dan bisa dipanggil dengan namespace view yang berbeda. Autentikasi tetap berada di luar modul, sedangkan otorisasi memakai kolom role, enum UserRole, helper isAdmin(), dan middleware EnsureUserHasRole.

Halaman Kelola User hanya bisa dibuka admin, memakai refresh halaman, dan mendukung pencarian, filter role, pagination, serta pengaman agar admin tidak menghapus atau menurunkan role akunnya sendiri. Modul Pendaftaran menampilkan, menyimpan, mengubah, dan menghapus data lewat AJAX tanpa memuat ulang halaman, termasuk unggah berkas lewat FormData dan pesan validasi per input. Catatan yang perlu ditindaklanjuti adalah tangkapan layar pengujian. Aplikasi sudah bisa dijalankan, tetapi bukti visualnya harus dilengkapi dengan menangkap layar tiap halaman yang disebutkan di Bab 4.